

Ugrađeni sustavi

Sveučilište J. J. Strossmayera u Osijeku
Fakultet primijenjene matematike i informatike

Ultrazvučni radar za detekciju objekata s rotacijskim mehanizmom

Osijek, 2026.

Sadržaj

1. Uvod
2. Opis sustava
3. Logika ponašanja sustava
4. Tehničke specifikacije - Hardver
 1. Arduino Nano (ATmega328P)
 2. Ultrazvučni senzor HC-SR04
 3. Servo motor MG90S
 4. OLED zaslon 0.96"
 5. Shift registar 74HC595
 6. Piezo Buzzer i LED indikatori
5. Softversko okruženje i alati
6. Detaljna analiza programskog koda
 1. Konfiguracija i definicije
 2. I2C komunikacija i OLED upravljanje
 3. Upravljanje Shift registrom
 4. Mjerenje udaljenosti (Senzor)
 5. Glavna logika i vizualizacija
 6. Main funkcija i PWM konfiguracija
7. Zaključak
8. Literatura

1. Uvod

Razvoj suvremenih ugrađenih sustava omogućio je široku primjenu senzora u svrhu automatizacije i povećanja sigurnosti. Jedan od ključnih aspekata u robotici i sustavima pomoći pri vožnji je precizna detekcija prepreka i mjerenje udaljenosti.

Sustav je temeljen na Arduino Nano mikrokontroleru koji, koristeći HC-SR04 ultrazvučni senzor, prikuplja podatke o objektima u vidnom polju od 180 stupnjeva. Cilj projekta je integrirati elektromehaničku komponentu, poput servo motora, sa senzorskom jedinicom te prikazati dobivene rezultate na OLED zaslonu.

Osim tehničke izvedbe mjerenja, projekt demonstrira i rješenje za proširenje izlaznih kapaciteta mikrokontrolera korištenjem 74HC595 shift registra. Na taj se način ostvaruje vizualna (LED diode) i zvučna (buzzer) signalizacija koja korisnika pravovremeno upozorava na blizinu prepreke. Kroz ovaj rad bit će prikazana cijela logika sustava, od inicijalizacije hardvera do finalne obrade podataka.

2. Opis sustava

Sustav je dizajniran kao autonomni ultrazvučni radar koji koristi Arduino Nano kao središnju upravljačku jedinicu. Glavna zadaća sustava je kontinuirano skeniranje prostora ispred senzora u luku od približno 180 stupnjeva. To se postiže montiranjem ultrazvučnog senzora HC-SR04 na osovinu MG90S servo motora.

Tijekom rotacije, mikrokontroler upravlja odašiljanjem ultrazvučnih impulsa i mjerenjem vremena potrebnog za njihov povratak nakon odbijanja od prepreke. Ti se podaci u realnom vremenu obrađuju i pretvaraju u udaljenost izraženu u centimetrima. Rezultati mjerenja prikazuju se na OLED zaslonu krupnim fontom, dok sustav istovremeno upravlja vizualnom i zvučnom signalizacijom putem 74HC595 shift registra i piezo zvučnika kako bi upozorio korisnika na blizinu prepreke.

3. Logika ponašanja sustava

Logika rada sustava temelji se na neprekidnoj petlji koja sinkronizira pokret, mjerenje i signalizaciju. Rad se može podijeliti u sljedeće korake:

- **Inicijalizacija i početno pozicioniranje:** Pri pokretanju, sustav postavlja smjerove pinova, inicijalizira I2C komunikaciju za OLED i postavlja servo motor u središnji položaj (SERVO_MID).
- **Skeniranje (Sekvenca pokreta):** Servo motor se pomiče u četiri faze: od sredine prema desno, od desna natrag prema sredini, od sredine prema lijevo, te s lijeva natrag u sredinu. Svaki korak kuta prati kratka pauza od 80 ms kako bi se senzor stabilizirao prije mjerenja.
- **Proces mjerenja (Trigger-Echo):** Za svaki položaj motora, mikrokontroler šalje 10 μ s impuls na Trigger pin senzora. Zatim mjeri trajanje visokog napona na Echo pinu. Udaljenost se računa dijeljenjem vremena s faktorom 58 (vrijeme leta zvuka do prepreke i natrag).
- **Odlučivanje i signalizacija:** Na temelju izmjerene udaljenosti (d), sustav donosi odluke o stanju alarma:
 - Kritična zona ($d < 15$ cm): Aktivira se crvena LED dioda putem shift registra i uključuje se piezo buzzer neprekidnim tonom.
 - Zona upozorenja ($15 \text{ cm} \leq d \leq 30$ cm): Isključuje se buzzer, a aktivira se žuta LED dioda.
 - Sigurna zona ($d > 30$ cm): Isključuje se buzzer, a aktivira se zelena LED dioda, signalizirajući da nema prepreka u neposrednoj blizini.
- **Ažuriranje prikaza:** Nakon svakog mjerenja, OLED zaslon se osvježava novim podacima o udaljenosti.

4. Tehničke specifikacije - Hardver

4.1. Arduino Nano

Arduino Nano služi kao centralna jedinica za obradu podataka. Temelji se na ATmega328P mikrokontroleru koji radi na 16 MHz. Njegova uloga je generiranje preciznih vremenskih signala za ultrazvučni senzor, upravljanje PWM signalom za servo motor, te serijska komunikacija s OLED zaslonom i shift registrom.

4.2. Ultrazvučni senzor HC-SR04

Ovaj senzor radi na principu sonara. Odašilje 'ping' (40 kHz) i mjeri vrijeme jeke. Udaljenost se izračunava pomoću brzine zvuka u zraku (cca 343 m/s). Minimalna udaljenost mjerenja je 2 cm, dok je maksimalni teoretski doseg 4 metra.

4.3. Servo motor MG90S

Digitalni mikro servo s metalnim zupčanicima. Zahtijeva PWM signal frekvencije 50 Hz. Trajanje impulsa određuje kut zakreta ruke motora na kojoj je montiran senzor.

4.4. OLED zaslon 0.96"

Zaslon rezolucije 128x64 piksela. Komunicira putem I2C protokola na adresi 0x3C. Korišten je za ispis numeričke vrijednosti udaljenosti krupnim fontom.

4.5. Shift registar 74HC595

Koristi se za proširenje izlaznih kapaciteta mikrokontrolera. Serijski prima podatke (data, clock, latch) i pretvara ih u paralelne izlaze za upravljanje LED indikatorima.

4.6. Piezo Buzzer i LED indikatori

Komponente zadužene za zvučnu i svjetlosnu signalizaciju. Crvena, žuta i zelena LED dioda prikazuju razinu opasnosti, dok piezo buzzer emitira ton u kritičnoj zoni.

5. Softversko okruženje i alati

Razvoj softverskog dijela ovog projekta temelji se na AVR Toolchain-u, skupu specijaliziranih alata otvorenog koda koji omogućuju razvoj aplikacija za Atmel (sada Microchip) AVR mikrokontrolere u jeziku C, bez potrebe za korištenjem standardnog Arduino IDE sučelja ili biblioteka. Ovakav pristup omogućuje izravnu kontrolu nad registrima mikrokontrolera ATmega328P i rezultira optimiziranim i bržim kodom.

Glavni alati korišteni u ovom projektu su:

- **avr-gcc:** Riječ je o prilagođenoj verziji GCC (GNU Compiler Collection) kompajlera. Njegova je zadaća prevesti C izvorni kod (radar_avr.c) u izvršni strojni kod (.elf datoteka) optimiziran za 8-bitnu AVR arhitekturu.
- **avr-objcopy:** Ovaj alat služi za ekstrakciju podataka iz .elf datoteke dobivene kompajliranjem te njihovo pretvaranje u .hex format. HEX datoteka sadrži stvarne binarne podatke koji će biti upisani u flash memoriju mikrokontrolera.
- **avrdude (AVR Downloader UploaDER):** Ključni uslužni program koji se koristi za prijenos (flash) HEX datoteke s računala na mikrokontroler putem USB kabela, koristeći serijski protokol.
- **PowerShell (automatizacija):** Kako bi se ubrzao proces razvoja, kreirana je skripta pokreni.ps1. Ona automatizira cijeli proces – od provjere sintakse, preko kompajliranja i generiranja binarne datoteke, pa sve do samog uploada na Arduino Nano pritiskom na jednu tipku.
- **Visual Studio Code:** Korišten kao primarni editor koda zbog svoje podrške za isticanje sintakse, ugrađenog terminala i lake integracije s PowerShell skriptama.
-

PowerShell skripta za automatizaciju (pokreni.ps1):

```
$MOJ_PORT = "COM3"
$USER_HOME = $env:USERPROFILE
$STAZA_ALATA =
"$USER_HOME\AppData\Local\Arduino15\packages\arduino\tools"

$GCC = Get-ChildItem -Path "$STAZA_ALATA\avr-gcc\*\bin\avr-gcc.exe" |
Select-Object -ExpandProperty FullName -First 1
$OBJCOPY = Get-ChildItem -Path "$STAZA_ALATA\avr-gcc\*\bin\avr-
objcopy.exe" | Select-Object -ExpandProperty FullName -First 1
$AVRDUDE = Get-ChildItem -Path
"$STAZA_ALATA\avrdude\*\bin\avrdude.exe" | Select-Object -
ExpandProperty FullName -First 1
$CONF = Get-ChildItem -Path "$STAZA_ALATA\avrdude\*\etc\avrdude.conf"
| Select-Object -ExpandProperty FullName -First 1

Clear-Host
Write-Host "--- KREĆEM S RADOM ---" -ForegroundColor Cyan

Write-Host "1. Kompajliram kod..." -ForegroundColor White
& $GCC -Wall -Os -mmcu=atmega328p radar_avr.c -o radar.elf
if ($LASTEXITCODE -ne 0) { Write-Host "GREŠKA: Kompajliranje nije
uspjelo!" -ForegroundColor Red; exit }
```

```
Write-Host "2. Generiram HEX datoteku..." -ForegroundColor White  
& $OBJCOPY -O ihex radar.elf radar.hex
```

```
Write-Host "3. Šaljem na Arduino na portu $MOJ_PORT..." -  
ForegroundColor Yellow  
& $AVRDUDE -C "$CONF" -v -patmega328p -carduino -P $MOJ_PORT -b 115200  
-D -U flash:w:radar.hex:i
```

```
if ($LASTEXITCODE -eq 0) {  
    Write-Host "`n--- USPJEH! RADAR JE POKRENUT ---" -ForegroundColor  
Green  
} else {  
    Write-Host "`n--- GREŠKA PRI SLANJU! Provjeri COM port i kabel. --  
-" -ForegroundColor Red  
}
```

6. Detaljna analiza programskog koda

6.1. Konfiguracija i definicije

Na početku koda definiramo frekvenciju takta i uključujemo potrebne zaglavlja za ulazno-izlazne operacije i kašnjenja.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>

#define OLED_ADDR 0x3C
#define SERVO_MID 22
#define SERVO_RIGHT 20
#define SERVO_LEFT 25
```

Ovdje su SERVO definicije (20, 22, 25) vrijednosti koje se upisuju u OCR2A registar kako bi se dobili odgovarajući kutovi.

6.2. I2C komunikacija i OLED upravljanje

I2C (Inter-Integrated Circuit) koristi dvije linije (SDA i SCL). TWBR registar postavlja brzinu prijenosa.

```
void i2c_init() { TWBR = 72; TWSR = 0; }
void i2c_start() { TWCR = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN); while
    (!(TWCR & (1<<TWINT))); }
void i2c_stop() { TWCR = (1<<TWINT) | (1<<TWSTO) | (1<<TWEN); }
void i2c_write(uint8_t data) { TWDR = data; TWCR = (1<<TWINT) |
    (1<<TWEN); while (!(TWCR & (1<<TWINT))); }

void oled_cmd(uint8_t cmd) {
    i2c_start();
    i2c_write(OLED_ADDR << 1);
    i2c_write(0x00);
    i2c_write(cmd);
    i2c_stop();
}

void oled_data(uint8_t data) {
    i2c_start();
    i2c_write(OLED_ADDR << 1);
    i2c_write(0x40);
    i2c_write(data);
    i2c_stop();
}
```

Oled_cmd šalje instrukcije kontroleru zaslona, dok oled_data ispisuje stvarne piksele u memoriju

zaslona.

6.3. Upravljanje Shift registrom

Funkcija `shift_out` serijski šalje 8 bita podataka u 74HC595 čip koji zatim paralelno uključuje LED diode.

```
void shift_out(uint8_t data) {
    PORTD &= ~(1 << 4); // Latch Low
    for (int i = 0; i < 8; i++) {
        if (data & (1 << (7 - i))) PORTD |= (1 << 2); else PORTD &=
~(1 << 2);
        PORTD |= (1 << 3); _delay_us(2); PORTD &= ~(1 << 3); // Clock
pulse
    }
    PORTD |= (1 << 4); // Latch High
}
```

6.4. Mjerenje udaljenosti (Senzor)

Senzor zahtijeva 10us impuls na Trigger pinu. Zatim mjerimo trajanje visokog nivoa na Echo pinu.

```
uint16_t get_dist() {
    PORTB &= ~(1 << 1); _delay_us(2);
    PORTB |= (1 << 1); _delay_us(10); // Trigger Pulse
    PORTB &= ~(1 << 1);
    uint32_t d = 0;
    while (!(PINB & (1 << 2)) & d < 20000) d++;
    d = 0;
    while (PINB & (1 << 2) & d < 20000) { d++; _delay_us(1); }
    return d / 58; // Konverzija u cm
}
```

6.5. Glavna logika i vizualizacija

Funkcija `skeniraj_i_ispisi()` objedinjuje očitavanje, ispis na ekran i zvučnu/svjetlosnu signalizaciju.

```
void skeniraj_i_ispisi() {
    uint16_t d = get_dist();
    if (d > 99) d = 99;
    // Brisanje starog ispisa i crtanje novih znamenki
    oled_print_digit_big((d/10)%10, 50);
    oled_print_digit_big(d%10, 62);
    if (d > 0 && d < 15) {
        shift_out(0b00000100); // Crvena LED
    }
}
```

```

        PORTB |= (1 << 0); // Buzzer
    } else {
        PORTB &= ~(1 << 0);
        if (d >= 15 && d <= 30) shift_out(0b00000010); // Žuta
        else shift_out(0b00000001); // Zelena
    }
}

```

6.6. Main funkcija i PWM konfiguracija

Ovdje se postavljaju DDR registri za ulaz/izlaz i konfigurira Timer2 za PWM upravljanje servom.

```

int main(void) {
    DDRB |= (1 << 1) | (1 << 0) | (1 << 3);
    DDRD |= (1 << 2) | (1 << 3) | (1 << 4);
    // Fast PWM mode, Phase Correct, 1024 prescaler
    TCCR2A = (1 << WGM21) | (1 << WGM20) | (1 << COM2A1);
    TCCR2B = (1 << CS22) | (1 << CS21) | (1 << CS20);
    i2c_init(); oled_init();
    while (1) {
        for (uint8_t pos = SERVO_MID; pos >= SERVO_RIGHT; pos--) {
            OCR2A = pos; skeniraj_i_ispisi(); _delay_ms(80); }
        for (uint8_t pos = SERVO_RIGHT; pos <= SERVO_MID; pos++) {
            OCR2A = pos; skeniraj_i_ispisi(); _delay_ms(80); }
        for (uint8_t pos = SERVO_MID; pos <= SERVO_LEFT; pos++) {
            OCR2A = pos; skeniraj_i_ispisi(); _delay_ms(80); }
        for (uint8_t pos = SERVO_LEFT; pos >= SERVO_MID; pos--) {
            OCR2A = pos; skeniraj_i_ispisi(); _delay_ms(80); }
    }
}

```

7. Zaključak

Kroz ovaj projekt uspješno je realiziran sustav ultrazvučnog radara s rotacijskim mehanizmom. Korištenjem AVR C programskog jezika i izravnim upravljanjem registrima postignuta je visoka učinkovitost i optimalno iskorištenje resursa ATmega328P mikrokontrolera. Sustav pruža pouzdanu detekciju prepreka u stvarnom vremenu s jasnom vizualnom, zvučnom i numeričkom signalizacijom, čime u potpunosti ispunjava zahtjeve naprednih ugrađenih sustava.

8. Literatura

- <https://docs.espressif.com/>
- [ATmega328P Datasheet](#)
- [GitHub Repository - AVR Examples](#)